数据整理

pandas 速查表
http://pandas.pydata.org

pandas API 参考　　pandas 用户指南

整洁数据——pandas 数据整理的基础

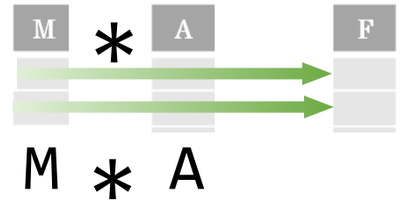
整洁数据集中：



每个变量存放 in its own **column**

每条观测记录存放在独立的行中

整洁数据适合 pandas 向量化运算。处理变量时 pandas 会保留观测记录；其他数据组织方式很难同样直观地使用 pandas。



创建 DataFrame

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
        {"a" : [4, 5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
       index = [1, 2, 3])
```
为每列指定数据

```
df = pd.DataFrame(
     [[4, 7, 10],
      [5, 8, 11],
      [6, 9, 12]],
     index=[1, 2, 3],
     columns=['a', 'b', 'c'])
```
为每行指定数据

		a	b	c
N	v			
D	1	4	7	10
	2	5	8	11
e	2	6	9	12

```
df = pd.DataFrame(
        {"a" : [4 ,5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
index = pd.MultiIndex.from_tuples(
       [('d', 1), ('d', 2),
        ('e', 2)], names=['n', 'v']))
```
创建多级索引 DataFrame

链式调用

多数 pandas 方法返回 DataFrame，
可在结果上继续调用其他方法
从而提高代码可读性

```
df = (pd.melt(df)
        .rename(columns={
                'variable':'var',
                'value':'val'})
        .query('val >= 200')
    )
```

数据重塑——布局、排序、重建索引与重命名



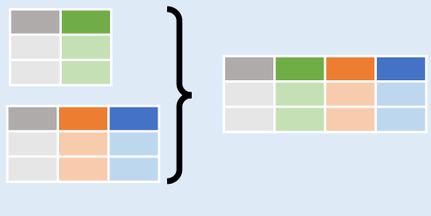
pd.melt(df)
将多列展开为行



df.pivot(columns='var', values='val')
将行透视为列



pd.concat([df1,df2])
纵向拼接 DataFrame



pd.concat([df1,df2], axis=1)
横向拼接 DataFrame

df.sort_values('mpg')
按列值升序排列

df.sort_values('mpg', ascending=False)
按列值降序排列

df.rename(columns = {'y':'year'})
重命名 DataFrame 的列

df.sort_index()
按索引排序

df.reset_index()
重置为默认行索引，原索引转为普通列

df.drop(columns=['Length', 'Height'])
删除指定列

选取行子集



df[df.Length > 7]
筛选满足条件的行

df.drop_duplicates()
删除重复行（依据列值）

df.sample(frac=0.5)
按比例随机抽取行

df.sample(n=10) 随机抽取 n 行

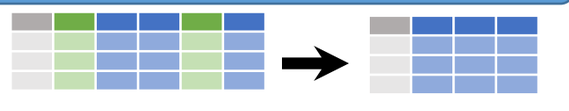
df.nlargest(n, 'value')
选择最大的 n 项并排序

df.nsmallest(n, 'value')
选择最小的 n 项并排序

df.head(n)
选取前 n 行

df.tail(n)
选取后 n 行

选取列子集



df[['width', 'length', 'species']]
按名称选择多列

df['width'] *or* **df.width**
按名称选择单列

df.filter(regex='*regex*')
选取名称满足正则表达式 regex 的列

使用 query

query() 支持用布尔表达式过滤行

```
df.query('Length > 7')
df.query('Length > 7 and Width < 8')
df.query('Name.str.startswith("abc")',
         engine="python")
```

选取行与列子集

使用 loc 和 iloc 可以选择行、列或二者
使用 at 和 iat 读取行列交叉处的单个值
第一维选行，第二维选列

df.iloc[10:20]
选择位置 10 至 19 的行

df.iloc[:, [1, 2, 5]]
选取位置 1、2、5 的列

（第一列位置为 0）

df.loc[:, 'x2':'x4']
选择 x2 至 x4 列（含两端）

df.loc[df['a'] > 10, ['a', 'c']]
选择满足条件的行，且只保留指定列

df.iat[1, 2] 按位置读取单个值
df.at[4, 'A'] 按标签读取单个值

Python（及 pandas）逻辑运算

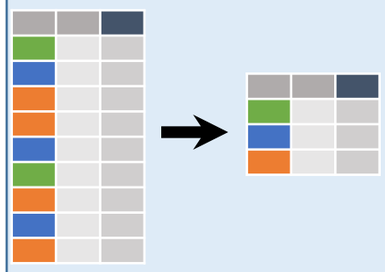
<	小于	!=	不等于
>	大于	df.column.isin(*values*)	判断是否属于集合
==	等于	pd.isnull(*obj*)	是否为缺失值
<=	小于等于	pd.notnull(*obj*)	是否非缺失值
>=	大于等于	&,|,~,^,df.any(),df.all()	与、或、非、异或、任意、全部

正则表达式示例

'\.'	匹配包含句点 . 的字符串
'Length$'	匹配以 Length 结尾的字符串
'^Sepal'	匹配以 Sepal 开头的字符串
'^x[1-5]$'	匹配 x 开头、1-5 结尾
'^(?!Species$).*'	匹配 Species 以外的字符串

Cheatsheet for pandas (http://pandas.pydata.org/ originally written by Irv Lustig, Princeton Consultants, inspired by Rstudio Data Wrangling Cheatsheet

数据分组



`df.groupby(by="col")`
返回 GroupBy 对象，按 col 列的值分组

`df.groupby(level="ind")`
返回 GroupBy 对象，按按索引层级中的值分组
索引层级名为 ind

以下方法同样适用于分组。此时按组执行，并返回与原 DataFrame 长度相同的结果向量

`shift(1)`
数值向下移动 1 位
`rank(method='dense')`
排名连续无间隔
`rank(method='min')`
并列值取最小名次
`rank(pct=True)`
将排名缩放到 [0,1]
`rank(method='first')`
并列时按出现顺序排名

`shift(-1)`
数值向上移动 1 位
`cumsum()`
累计求和
`cummax()`
累计最大值
`cummin()`
累计最小值
`cumprod()`
累计乘积

上述汇总函数均可用于分组
其他 GroupBy 函数：

`size()`
各组的大小

`agg(function)`
使用函数进行分组聚合

数据汇总

`df['w'].value_counts()`
统计各不同取值出现的次数
`len(df)`
DataFrame 行数
`df.shape`
返回（行数，列数）元组
`df['w'].nunique()`
列中不同值的数量
`df.describe()`
各列（或分组）的描述性统计
`df.info()`
输出 DataFrame 摘要信息
`df.memory_usage()`
输出各列内存占用
`df.dtypes`
返回包含各列数据类型的 Series



pandas 提供大量汇总函数，可用于多种 pandas 对象，包括列、Series、GroupBy、Expanding 和 Rolling，为每组生成单个值。用于 DataFrame 时各列的结果构成 Series，例如：

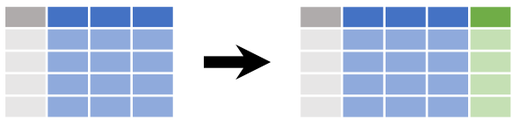
`sum()`
求和
`count()`
统计非空值的数量（每个对象）
`median()`
中位数
`quantile([0.25,0.75])`
分位数
`apply(function)`
对各对象应用函数

`min()`
最小值
`max()`
最大值
`mean()`
平均值
`var()`
方差
`std()`
标准差（每个对象）

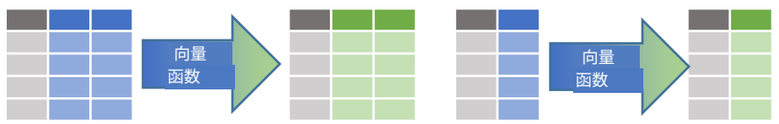
处理缺失值

`df.dropna()`
删除含有 NA/null 的行
`df.fillna(value)`
使用指定值填充 NA/null

新增列



`df.assign(Area=lambda df: df.Length*df.Height)`
计算并添加一列或多列
`df['Volume'] = df.Length*df.Height*df.Depth`
添加单列
`pd.qcut(df.col, n, labels=False)`
按分位数将列分成 n 组



pandas 提供大量向量函数，可作用于 DataFrame 全部列或某个选定列（Series），并为各列生成结果向量。例如：

`max(axis=1)`
逐行最大值
`clip(lower=-10,upper=10)`
把超出阈值的数截断至边界

`min(axis=1)`
逐行最小值
`abs()`
绝对值

窗口

`df.expanding()`
返回 Expanding 对象，以便累计应用汇总函数
`df.rolling(n)`
返回 Rolling 对象，以便在长度 n 的窗口应用汇总函数

合并数据集

adf

x1	x2
A	1
B	2
C	3

bdf

x1	x3
A	T
B	F
D	T

标准连接

x1	x2	x3
A	1	T
B	2	F
C	3	NaN

`pd.merge(adf, bdf, how='left', on='x1')`
以 adf 为主表进行左连接

x1	x2	x3
A	1.0	T
B	2.0	F
D	NaN	T

`pd.merge(adf, bdf, how='right', on='x1')`
以 bdf 为主表进行右连接

x1	x2	x3
A	1	T
B	2	F

`pd.merge(adf, bdf, how='inner', on='x1')`
内连接：只保留匹配的行

x1	x2	x3
A	1	T
B	2	F
C	3	NaN
D	NaN	T

`pd.merge(adf, bdf, how='outer', on='x1')`
外连接：保留两表全部行

过滤连接

x1	x2
A	1
B	2

`adf[adf.x1.isin(bdf.x1)]`
保留 adf 中与 bdf 匹配的行

x1	x2
C	3

`adf[~adf.x1.isin(bdf.x1)]`
保留 adf 中与 bdf 不匹配的行

ydf

x1	x2
A	1
B	2
C	3

zdf

x1	x2
B	2
C	3
D	4

类集合运算

x1	x2
B	2
C	3

`pd.merge(ydf, zdf)`
保留 ydf 与 zdf 中均出现的行（交集）

x1	x2
A	1
B	2
C	3
D	4

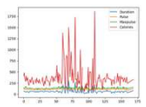
`pd.merge(ydf, zdf, how='outer')`
保留 ydf 或 zdf 中出现的行（并集）

x1	x2
A	1

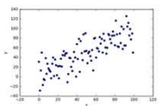
```
pd.merge(ydf, zdf, how='outer',
         indicator=True)
.query('_merge == "left_only"')
.drop(columns=['_merge'])
```
仅保留 ydf 中出现的行（差集）

绘图

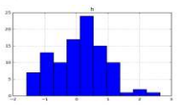
df.plot()
绘制折线图



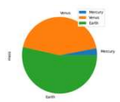
df.plot.scatter(x='w', y='h')
绘制散点图



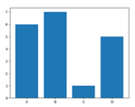
df.plot.hist()
绘制直方图



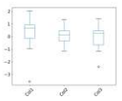
df.plot.pie()
绘制饼图



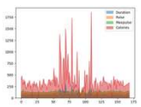
df.plot.bar()
绘制折线图



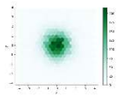
df.plot.boxplot()
绘制散点图



df.plot.area()
绘制面积图



df.plot.hexbin()
绘制六边形分箱图



df.plot(subplots=True)
将各列分别绘制成
独立图表

df.plot(cumulative=True)
绘制累计图

df.plot(stacked=True)
堆叠各列数据
（仅适用于 bar/barh/area）

df.plot(title="Graph of A against B")
设置图表标题

df.plot(bins=30)
设置数据分箱数量
（直方图）

df.plot(alpha=0.5)
设置图表透明度为 50%

df.plot(subplots=True, title=['col1', 'col2', 'col3'])
参数可组合使用：例如三列数据可分别绘图；标题列表的首项用于第一列
后续标题依次对应各列

转换数据类型

pd.to_numeric(data)
将非数值类型转换为
数值类型

pd.to_datetime(data)
将其他类型转换为
日期时间类型

pd.to_timedelta(data)
将其他类型转换为
时间差类型

df.astype(type)
将数据转换为指定类型，
包括分类类型

df.infer_objects()
尝试为 object 数据
推断更合适的类型

df.convert_dtypes()
将各列转换为更合适的
数据类型

日期时间

对于日期时间类型的 Series，
使用 dt 访问器提取
日期时间的
各个组成部分：

s.dt.year
提取年份

s.dt.month
提取月份（整数）

s.dt.day
提取日期中的日（整数）

s.dt.quarter
提取所属季度

s.dt.hour
提取小时

s.dt.minute
提取分钟

s.dt.second
提取秒

映射

对 DataFrame 或 Series 的每个元素
执行映射，用于重新分类和转换

s.map(lambda x: 2*x)
返回各元素乘 2 的 Series 副本

df.apply(lambda s: s.max() - s.min(), axis=1)
返回每一行最大值与最小值之差
（Series 形式）

Series 字符串操作

类似 Python 字符串操作，但进行向量化处理，
可高效作用于整个 Series

s.str.count(pattern)
返回每个元素中模式出现的
次数（整数）

s.str.get(index)
返回各字符串中指定
索引位置的字符

s.str.join(sep)
返回对各元素执行
拼接的 Series

s.str.title()
将每个单词的首字母
转换为大写

s.str.len()
返回各字符串的
长度

s.str.cat()
将所有元素拼接成
单个字符串

s.str.partition(sep)
按第一次出现的分隔符
切分字符串

s.str.slice(start, stop, step)
对各字符串切片

s.str.replace(pat, rep)
用正则表达式替换各
字符串中的匹配内容

s.str.isalnum()
判断各元素是否为
字母或数字字符

输入／输出

常见输入格式有 CSV、JSON、HTML 等
易读格式；输出常采用针对性能和
扩展性优化的 Feather、Parquet、HDF

df = pd.read_csv(filepath)
读取 CSV 文件

df = pd.read_html(filepath)
读取 HTML 文件

df = pd.read_excel(filepath)
读取 Excel 文件

df = pd.read_sql(filepath)
读取 SQL 数据

pd.read_clipboard()
读取剪贴板文本

df.to_parquet(filepath)
写入 Parquet 文件

df.to_feather(filepath)
写入 Feather 文件

df.to_hdf(filepath)
写入 HDF 文件

df.to_clipboard()
复制到系统剪贴板

常用功能选项

pandas 提供全局选项，
控制运行行为和显示等
可以通过以下方式查询与设置：
pd.options.option_name（其中
option_name 是选项名称），
例如：

pd.options.display.max_rows = 20
将最多显示行数设为 20

函数

get_option(option)
获取指定选项的值

set_option(option)
设置指定选项的值

reset_option(options)
将指定选项恢复为
默认值

describe_option(options)
输出指定选项的说明

option_context(options)
临时选项设置下执行代码
执行结束后恢复原来的
选项设置

显示选项

display.max_rows
最大显示行数
（格式化输出）

display.max_columns
最大显示列数
（格式化输出）

display.expand_frame_repr
控制 DataFrame 的
显示是否跨行展开

display.large_repr
控制超出最大行列数时
DataFrame 的呈现方式：
截断或汇总

display.precision
输出显示的小数
精度位数

display.max_colwidth
单元格显示的最大宽度，
过长内容将截断

display.max_info_columns
最大显示列数
调用 info() 时显示的最大列数

display.chop_threshold
显示时将足够接近零的数
视为零

display.colheader_justify
控制列标题的对齐方式